

Technical Requirements Document (TRD)

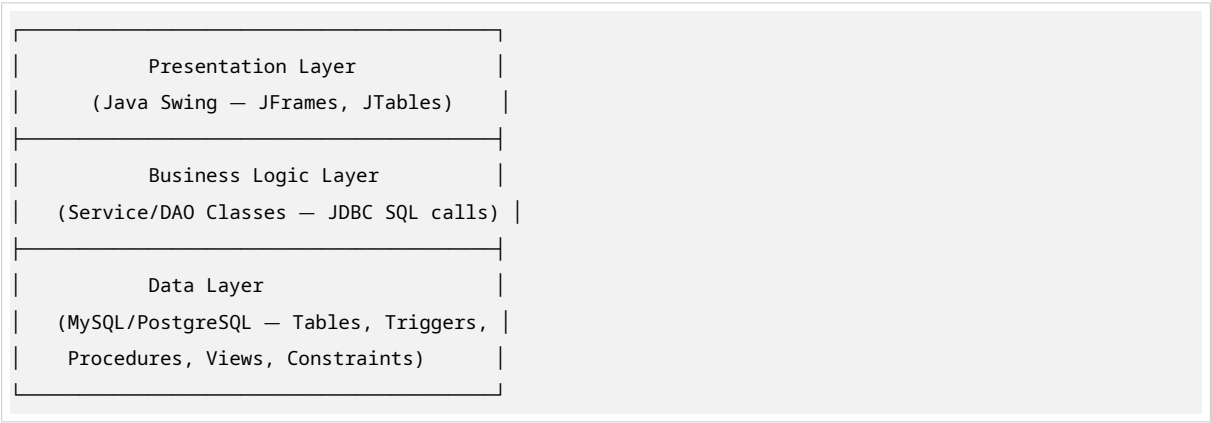
Hospital Bed Allocation System

Version: 1.0
Date: 2026-05-17
Status: Draft
Author: Student Project Team
Target: College DBMS Course Project

1. Architecture Overview

1.1 Architectural Pattern

Layered Architecture (3-Tier Desktop)



1.2 Design Patterns Used

- **DAO (Data Access Object):** Encapsulates all database access logic.
- **Singleton:** Database connection manager (DBConnection.java).
- **Model/POJO:** Plain Java objects mirroring database tables.
- **Observer:** Swing ActionListener for GUI event handling.

2. Technology Stack

Layer	Technology	Version	Purpose
Language	Java	JDK 17+	Core application logic
GUI Framework	Java Swing	Built-in	Desktop user interface
Database	MySQL	8.0+	Primary RDBMS (PostgreSQL 14+ acceptable)
DB Driver	JDBC (mysql-connector-j)	8.0+	Java-Database connectivity
Build Tool	Maven	3.9+	Dependency management & build

Table 1 – continued

Layer	Technology	Version	Purpose
IDE	IntelliJ IDEA / Eclipse	Latest	Development environment
DB Client	MySQL Workbench / pgAdmin	Latest	Schema design & testing

3. Database Design

3.1 Entity-Relationship Diagram (Logical)

```

[WARD] 1-----* [BED] 1-----* [ADMISSION] *-----1 [PATIENT]
|
|
|
1-----* [BED_MAINTENANCE_LOG]
|
[PATIENT] 1-----* [WAITLIST]
|
*-----1 [STAFF] (Doctor)

```

Cardinality:

- Ward → Bed: **1:N**
- Bed → Admission: **1:N** (historical admissions over time)
- Patient → Admission: **1:N**
- Staff → Admission: **1:N** (one doctor per admission)
- Bed → BedMaintenanceLog: **1:N**
- Patient → Waitlist: **1:N** (patient may wait multiple times historically)

3.2 Physical Schema (MySQL DDL)

```

-- =====
-- 3.2.1 WARDS TABLE
-- =====
CREATE TABLE wards (
  ward_id      INT PRIMARY KEY AUTO_INCREMENT,
  ward_type    VARCHAR(20) NOT NULL,
  floor        INT NOT NULL,
  capacity     INT NOT NULL CHECK (capacity > 0),
  daily_charge DECIMAL(10,2) NOT NULL CHECK (daily_charge >= 0),

  CONSTRAINT chk_ward_type CHECK (ward_type IN ('General','ICU','Emergency','Private'))
);

-- =====
-- 3.2.2 BEDS TABLE
-- =====

```

```

CREATE TABLE beds (
    bed_id          INT PRIMARY KEY AUTO_INCREMENT,
    ward_id         INT NOT NULL,
    bed_number      VARCHAR(10) NOT NULL,
    equipment_status VARCHAR(50) DEFAULT 'Standard',
    current_status  VARCHAR(20) DEFAULT 'Available',

    CONSTRAINT fk_bed_ward FOREIGN KEY (ward_id) REFERENCES wards(ward_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_bed_status CHECK (current_status IN ('Available','Occupied','Maintenance')),
    CONSTRAINT uq_bed_ward UNIQUE (ward_id, bed_number)
);

-- =====
-- 3.2.3 PATIENTS TABLE
-- =====

CREATE TABLE patients (
    patient_id      INT PRIMARY KEY AUTO_INCREMENT,
    name            VARCHAR(100) NOT NULL,
    age            INT CHECK (age >= 0 AND age <= 150),
    blood_group     VARCHAR(5),
    contact         VARCHAR(15) NOT NULL,
    address         VARCHAR(255),
    emergency_contact VARCHAR(15),
    registered_date  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- 3.2.4 STAFF TABLE
-- =====

CREATE TABLE staff (
    staff_id  INT PRIMARY KEY AUTO_INCREMENT,
    name     VARCHAR(100) NOT NULL,
    role     VARCHAR(20) NOT NULL,
    department VARCHAR(50),
    phone    VARCHAR(15),

    CONSTRAINT chk_staff_role CHECK (role IN ('Doctor','Nurse','Admin'))
);

-- =====
-- 3.2.5 ADMISSIONS TABLE
-- =====

CREATE TABLE admissions (
    admission_id      INT PRIMARY KEY AUTO_INCREMENT,
    patient_id        INT NOT NULL,
    bed_id            INT NOT NULL,
    doctor_id         INT,
    admission_date     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expected_discharge DATE,
    actual_discharge   TIMESTAMP NULL,

```

```

status          VARCHAR(20) DEFAULT 'Active',

CONSTRAINT fk_adm_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
CONSTRAINT fk_adm_bed FOREIGN KEY (bed_id) REFERENCES beds(bed_id),
CONSTRAINT fk_adm_doctor FOREIGN KEY (doctor_id) REFERENCES staff(staff_id),
CONSTRAINT chk_adm_status CHECK (status IN ('Active','Discharged','Transferred')),

-- Ensure a bed cannot have two active admissions
CONSTRAINT uq_active_bed UNIQUE (bed_id, status)
);

-- =====
-- 3.2.6 WAITLIST TABLE
-- =====
CREATE TABLE waitlist (
    waitlist_id      INT PRIMARY KEY AUTO_INCREMENT,
    patient_id       INT NOT NULL,
    requested_ward_type VARCHAR(20) NOT NULL,
    priority_score    INT NOT NULL DEFAULT 0,
    request_time      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status            VARCHAR(20) DEFAULT 'Waiting',

    CONSTRAINT fk_wl_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    CONSTRAINT chk_wl_status CHECK (status IN ('Waiting','Allocated','Cancelled')),
    CONSTRAINT chk_wl_ward CHECK (requested_ward_type IN ('General','ICU','Emergency','Private'))
);

-- =====
-- 3.2.7 BED MAINTENANCE LOG TABLE
-- =====
CREATE TABLE bed_maintenance_log (
    log_id          INT PRIMARY KEY AUTO_INCREMENT,
    bed_id          INT NOT NULL,
    start_time       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    end_time         TIMESTAMP NULL,
    reason           VARCHAR(255) NOT NULL,
    resolved_by      INT,

    CONSTRAINT fk_maint_bed FOREIGN KEY (bed_id) REFERENCES beds(bed_id),
    CONSTRAINT fk_maint_staff FOREIGN KEY (resolved_by) REFERENCES staff(staff_id)
);

-- =====
-- 3.2.8 BED STATUS AUDIT LOG TABLE
-- =====
CREATE TABLE bed_status_audit (
    audit_id        INT PRIMARY KEY AUTO_INCREMENT,
    bed_id          INT NOT NULL,
    old_status       VARCHAR(20),
    new_status       VARCHAR(20),
    changed_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

```
changed_by    VARCHAR(50),
reason        VARCHAR(255)
);
```

3.3 Normalization Trace

UNF (Unnormalized Form) Monster Table: hospital_records(patient_id, patient_name, age, ward_type, bed_no, bed_equipment, doctor_name, nurse_name, admission_date, discharge_date, medicines)

Issues:

- Repeating groups: multiple medicines, multiple doctors per admission.
- Multi-valued attributes: bed_equipment contains comma-separated values.
- Insertion anomaly: Cannot add a new bed without a patient.
- Deletion anomaly: Deleting a patient loses bed information.
- Update anomaly: Changing ward floor requires updating all bed records.

1NF

- Atomic values: Split medicines into separate table.
- Separate bed_equipment into single-value entries or separate table.
- Remove repeating groups: Doctor and Nurse assignments in separate rows.

Result Tables: patients, beds, admissions, medicines, staff_assignments

2NF Partial Dependencies Identified:

- bed_no → ward_type, bed_equipment (depends on part of composite key patient_id + bed_no + admission_date)
- doctor_name → doctor_department (depends on doctor, not admission)

Decomposition:

- Extract beds(bed_id, ward_id, bed_number, equipment_status)
- Extract wards(ward_id, ward_type, floor, capacity, daily_charge)
- Extract staff(staff_id, name, role, department)

3NF Transitive Dependencies Identified:

- ward_id → ward_type → daily_charge (ward_type determines charge, not bed)
- patient_id → address, contact (contact info depends on patient, not admission)

Already resolved in 2NF decomposition. Final schema is in 3NF.

BCNF Check: Every determinant is a candidate key.

- bed_id is PK of beds — determines all non-key attributes.
- ward_id is PK of wards — determines all ward attributes.

- patient_id is PK of patients — determines all patient attributes.
- admission_id is PK of admissions — determines all admission attributes.

Result: Schema satisfies BCNF.

3.4 Functional Dependencies Documented

Table	FD	Type
wards	ward_id → ward_type, floor, capacity, daily_charge	PK dependency
beds	bed_id → ward_id, bed_number, equipment_status, current_status	PK dependency
patients	patient_id → name, age, blood_group, contact, address	PK dependency
staff	staff_id → name, role, department, phone	PK dependency
admissions	admission_id → patient_id, bed_id, doctor_id, dates, status	PK dependency
admissions	bed_id + status='Active' → admission_id, patient_id	Unique constraint (one active admission per bed)
waitlist	waitlist_id → patient_id, requested_ward_type, priority_score, request_time	PK dependency

4. Database Objects (Advanced DBMS Concepts)

4.1 Triggers

```
-- TRIGGER 1: Auto-update bed status on admission
DELIMITER //
CREATE TRIGGER trg_bed_occupied_after_admission
AFTER INSERT ON admissions
FOR EACH ROW
BEGIN
    IF NEW.status = 'Active' THEN
        UPDATE beds SET current_status = 'Occupied' WHERE bed_id = NEW.bed_id;

        INSERT INTO bed_status_audit (bed_id, old_status, new_status, changed_by, reason)
        SELECT NEW.bed_id, current_status, 'Occupied', 'SYSTEM', 'Patient Admission'
        FROM beds WHERE bed_id = NEW.bed_id;
    END IF;
END//
DELIMITER ;

-- TRIGGER 2: Auto-update bed status on discharge
```

```

DELIMITER //
CREATE TRIGGER trg_bed_freed_after_discharge
AFTER UPDATE ON admissions
FOR EACH ROW
BEGIN
    IF OLD.status = 'Active' AND NEW.status = 'Discharged' THEN
        UPDATE beds SET current_status = 'Available' WHERE bed_id = NEW.bed_id;

        INSERT INTO bed_status_audit (bed_id, old_status, new_status, changed_by, reason)
        VALUES (NEW.bed_id, 'Occupied', 'Available', 'SYSTEM', 'Patient Discharge');
    END IF;
END//
DELIMITER ;

```

4.2 Stored Procedures

```

-- PROCEDURE 1: Allocate bed atomically
DELIMITER //
CREATE PROCEDURE sp_allocate_bed(
    IN p_patient_id INT,
    IN p_ward_type VARCHAR(20),
    IN p_doctor_id INT,
    IN p_expected_discharge DATE,
    OUT p_bed_id INT,
    OUT p_admission_id INT,
    OUT p_success BOOLEAN
)
BEGIN
    DECLARE v_bed_id INT DEFAULT NULL;
    DECLARE v_admission_id INT DEFAULT NULL;

    START TRANSACTION;

    -- Find first available bed in requested ward type
    SELECT b.bed_id INTO v_bed_id
    FROM beds b
    JOIN wards w ON b.ward_id = w.ward_id
    WHERE w.ward_type = p_ward_type
        AND b.current_status = 'Available'
        AND b.equipment_status != 'Broken'
    ORDER BY b.bed_id
    LIMIT 1
    FOR UPDATE; -- Pessimistic lock

    IF v_bed_id IS NOT NULL THEN
        -- Create admission
        INSERT INTO admissions (patient_id, bed_id, doctor_id, expected_discharge, status)
        VALUES (p_patient_id, v_bed_id, p_doctor_id, p_expected_discharge, 'Active');

        SET v_admission_id = LAST_INSERT_ID();
    END IF;
END//
DELIMITER ;

```

```

-- Update bed status (also handled by trigger, but explicit for safety)
UPDATE beds SET current_status = 'Occupied' WHERE bed_id = v_bed_id;

SET p_bed_id = v_bed_id;
SET p_admission_id = v_admission_id;
SET p_success = TRUE;

COMMIT;
ELSE
-- No bed available — add to waitlist
INSERT INTO waitlist (patient_id, requested_ward_type, priority_score, status)
VALUES (p_patient_id, p_ward_type, 100, 'Waiting');

SET p_success = FALSE;
SET p_bed_id = NULL;
SET p_admission_id = NULL;

COMMIT;
END IF;
END//
DELIMITER ;

-- PROCEDURE 2: Discharge patient and process waitlist
DELIMITER //
CREATE PROCEDURE sp_discharge_patient(
    IN p_admission_id INT,
    IN p_maintenance_needed BOOLEAN,
    OUT p_message VARCHAR(255)
)
BEGIN
    DECLARE v_bed_id INT;
    DECLARE v_ward_id INT;
    DECLARE v_ward_type VARCHAR(20);
    DECLARE v_waitlist_patient INT;
    DECLARE v_waitlist_id INT;

    START TRANSACTION;

    -- Get bed and ward info
    SELECT a.bed_id, b.ward_id, w.ward_type
    INTO v_bed_id, v_ward_id, v_ward_type
    FROM admissions a
    JOIN beds b ON a.bed_id = b.bed_id
    JOIN wards w ON b.ward_id = w.ward_id
    WHERE a.admission_id = p_admission_id AND a.status = 'Active';

    -- Update admission
    UPDATE admissions
    SET status = 'Discharged', actual_discharge = NOW()
    WHERE admission_id = p_admission_id;

```

```

-- Update bed status
IF p_maintenance_needed THEN
    UPDATE beds SET current_status = 'Maintenance' WHERE bed_id = v_bed_id;
    SET p_message = 'Patient discharged. Bed marked for maintenance.';
ELSE
    UPDATE beds SET current_status = 'Available' WHERE bed_id = v_bed_id;

    -- Check waitlist for auto-allocation
    SELECT wl.waitlist_id, wl.patient_id
    INTO v_waitlist_id, v_waitlist_patient
    FROM waitlist wl
    WHERE wl.requested_ward_type = v_ward_type
        AND wl.status = 'Waiting'
    ORDER BY wl.priority_score DESC, wl.request_time ASC
    LIMIT 1;

    IF v_waitlist_id IS NOT NULL THEN
        -- Auto-allocate to waitlisted patient
        UPDATE waitlist SET status = 'Allocated' WHERE waitlist_id = v_waitlist_id;

        INSERT INTO admissions (patient_id, bed_id, status, admission_date)
        VALUES (v_waitlist_patient, v_bed_id, 'Active', NOW());

        UPDATE beds SET current_status = 'Occupied' WHERE bed_id = v_bed_id;

        SET p_message = CONCAT('Patient discharged. Bed auto-allocated to waitlisted patient ID: ',
            ↪ v_waitlist_patient);
    ELSE
        SET p_message = 'Patient discharged. Bed is now available.';
    END IF;
END IF;

COMMIT;
END//
DELIMITER ;

```

4.3 Views

```

-- VIEW 1: Current Occupancy Dashboard
CREATE VIEW vw_current_occupancy AS
SELECT
    w.ward_type,
    COUNT(b.bed_id) AS total_beds,
    SUM(CASE WHEN b.current_status = 'Occupied' THEN 1 ELSE 0 END) AS occupied,
    SUM(CASE WHEN b.current_status = 'Available' THEN 1 ELSE 0 END) AS available,
    SUM(CASE WHEN b.current_status = 'Maintenance' THEN 1 ELSE 0 END) AS maintenance,
    ROUND(SUM(CASE WHEN b.current_status = 'Occupied' THEN 1 ELSE 0 END) * 100.0 / COUNT(b.bed_id), 2)
    ↪ AS occupancy_rate
FROM wards w
LEFT JOIN beds b ON w.ward_id = b.ward_id
GROUP BY w.ward_type;

```

```

-- VIEW 2: Active Admissions with Details
CREATE VIEW vw_active_admissions AS
SELECT
    a.admission_id,
    p.name AS patient_name,
    p.age,
    p.blood_group,
    w.ward_type,
    b.bed_number,
    s.name AS doctor_name,
    a.admission_date,
    a.expected_discharge,
    DATEDIFF(COALESCE(a.actual_discharge, NOW()), a.admission_date) AS days_admitted
FROM admissions a
JOIN patients p ON a.patient_id = p.patient_id
JOIN beds b ON a.bed_id = b.bed_id
JOIN wards w ON b.ward_id = w.ward_id
LEFT JOIN staff s ON a.doctor_id = s.staff_id
WHERE a.status = 'Active';

-- VIEW 3: Available Beds
CREATE VIEW vw_available_beds AS
SELECT
    b.bed_id,
    w.ward_type,
    w.floor,
    b.bed_number,
    b.equipment_status,
    w.daily_charge
FROM beds b
JOIN wards w ON b.ward_id = w.ward_id
WHERE b.current_status = 'Available'
ORDER BY w.ward_type, b.bed_number;

-- VIEW 4: Waitlist with Patient Details
CREATE VIEW vw_waitlist_details AS
SELECT
    wl.waitlist_id,
    p.name AS patient_name,
    p.age,
    p.contact,
    wl.requested_ward_type,
    wl.priority_score,
    wl.request_time,
    TIMESTAMPDIFF(HOUR, wl.request_time, NOW()) AS waiting_hours,
    wl.status
FROM waitlist wl
JOIN patients p ON wl.patient_id = p.patient_id
WHERE wl.status = 'Waiting'
ORDER BY wl.priority_score DESC, wl.request_time ASC;

```

```
-- VIEW 5: Bed Turnover Report
CREATE VIEW vw_bed_turnover AS
SELECT
    w.ward_type,
    COUNT(a.admission_id) AS total_admissions,
    AVG(DATEDIFF(a.actual_discharge, a.admission_date)) AS avg_stay_days,
    MAX(DATEDIFF(a.actual_discharge, a.admission_date)) AS max_stay_days,
    MIN(DATEDIFF(a.actual_discharge, a.admission_date)) AS min_stay_days
FROM admissions a
JOIN beds b ON a.bed_id = b.bed_id
JOIN wards w ON b.ward_id = w.ward_id
WHERE a.status = 'Discharged' AND a.actual_discharge IS NOT NULL
GROUP BY w.ward_type;
```

4.4 Indexes

```
-- Performance indexes for frequent queries
CREATE INDEX idx_bed_status ON beds(current_status);
CREATE INDEX idx_bed_ward ON beds(ward_id, current_status);
CREATE INDEX idx_adm_status ON admissions(status);
CREATE INDEX idx_adm_bed_active ON admissions.bed_id, status);
CREATE INDEX idx_waitlist_status ON waitlist(status, requested_ward_type);
CREATE INDEX idx_waitlist_priority ON waitlist(priority_score DESC, request_time ASC);
CREATE INDEX idx_patient_name ON patients(name);
```

4.5 Java Application Architecture

4.5.1 Package Structure

```
com.hospital.bedalloc
├─ db
│   ├── DBConnection.java          # Singleton JDBC connection manager
│   └─ DatabaseConfig.java         # Connection parameters (externalized)
├─ dao
│   ├── WardDAO.java
│   ├── BedDAO.java
│   ├── PatientDAO.java
│   ├── AdmissionDAO.java
│   ├── WaitlistDAO.java
│   ├── StaffDAO.java
│   └─ ReportDAO.java
├─ model
│   ├── Ward.java
│   ├── Bed.java
│   ├── Patient.java
│   ├── Admission.java
│   ├── Waitlist.java
│   └─ Staff.java
```

```

|   └─ BedStatusAudit.java
└─ service
|   └─ BedAllocationService.java # Business logic / transaction coordinator
|   └─ DischargeService.java
└─ gui
|   └─ MainFrame.java           # Primary dashboard
|   └─ LoginFrame.java
|   └─ DashboardPanel.java
|   └─ AdmitPatientDialog.java
|   └─ DischargeDialog.java
|   └─ WaitlistPanel.java
|   └─ ReportsFrame.java
|   └─ AdminPanel.java
|   └─ components
|       └─ BedStatusRenderer.java # Custom JTable cell renderer
|       └─ StatusBadge.java
└─ util
    └─ DateUtil.java
    └─ ValidationUtil.java
    └─ SwingWorkerUtil.java

```

4.5.2 Key Class Specifications

DBConnection.java

```

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/hospital_db";
    private static final String USER = "root";
    private static final String PASSWORD = "password"; // Externalize in properties file
    private static Connection connection;

    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            connection = DriverManager.getConnection(URL, USER, PASSWORD);
        }
        return connection;
    }

    public static void closeConnection() {
        if (connection != null) {
            try { connection.close(); } catch (SQLException e) { e.printStackTrace(); }
        }
    }
}

```

BedDAO.java (Core Methods)

```

public class BedDAO {

    // Retrieve all beds with ward info for dashboard
    public List<Bed> getAllBedsWithWard() throws SQLException;
}

```

```

// Find available bed by ward type (used in allocation)
public Bed findAvailableBed(String wardType) throws SQLException;

// Update bed status (Available, Occupied, Maintenance)
public boolean updateBedStatus(int bedId, String newStatus) throws SQLException;

// Get bed count by status per ward
public Map<String, int[]> getBedStatistics() throws SQLException;
}

```

AdmissionDAO.java (Core Methods)

```

public class AdmissionDAO {

    // Call stored procedure sp_allocate_bed
    public AllocationResult allocateBed(int patientId, String wardType,
                                        int doctorId, Date expectedDischarge) throws SQLException;

    // Call stored procedure sp_discharge_patient
    public String dischargePatient(int admissionId, boolean maintenanceNeeded) throws SQLException;

    // Get all active admissions (for discharge panel)
    public List<Admission> getActiveAdmissions() throws SQLException;

    // Get admission history for a patient
    public List<Admission> getPatientAdmissionHistory(int patientId) throws SQLException;
}

```

BedStatusRenderer.java (Swing Custom Component)

```

public class BedStatusRenderer extends DefaultTableCellRenderer {
    @Override
    public Component getTableCellRendererComponent(JTable table, Object value,
                                                    boolean isSelected, boolean hasFocus, int row, int column) {
        Component c = super.getTableCellRendererComponent(...);
        String status = (String) value;
        switch(status) {
            case "Available": c.setBackground(Color.GREEN); break;
            case "Occupied": c.setBackground(Color.RED); break;
            case "Maintenance": c.setBackground(Color.YELLOW); break;
            default: c.setBackground(Color.WHITE);
        }
        return c;
    }
}

```

4.5.3 Transaction Handling

All allocation and discharge operations must use **JDBC transactions**:

```
Connection conn = DBConnection.getConnection();
conn.setAutoCommit(false);
try {
    // Step 1: Find available bed (FOR UPDATE lock)
    // Step 2: Insert admission record
    // Step 3: Update bed status
    // Step 4: If applicable, update waitlist
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
    throw e;
} finally {
    conn.setAutoCommit(true);
}
```

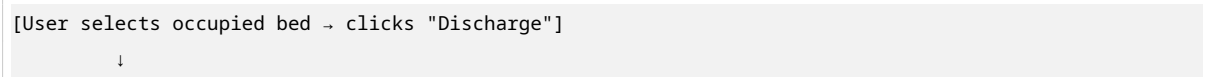
Isolation Level: REPEATABLE READ or SERIALIZABLE for bed allocation to prevent phantom reads and double-booking.

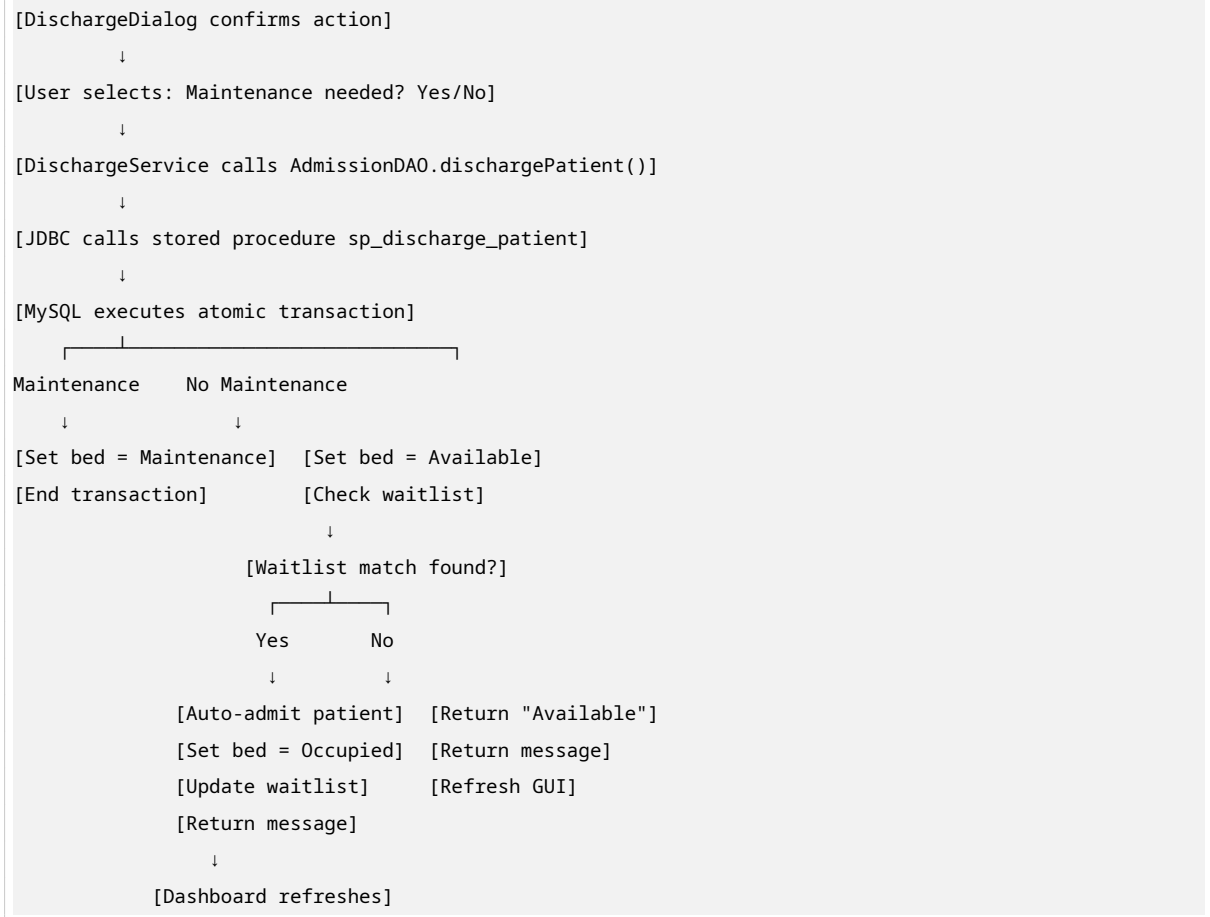
5. Data Flow Diagrams

5.1 Patient Admission Flow



5.2 Patient Discharge Flow





6. Security Requirements

ID	Requirement
SEC-01	Database credentials stored in external <code>config.properties</code> , not hardcoded
SEC-02	SQL Injection prevention via <code>PreparedStatement</code> for all user inputs
SEC-03	Input validation on all Swing forms (non-empty checks, type validation, length limits)
SEC-04	Basic login screen with hardcoded admin/receptionist roles (college scope)
SEC-05	No sensitive patient data (SSN, financial info) stored

7. Performance Requirements

ID	Requirement	Target
PERF-01	Dashboard JTable load time	< 2 seconds
PERF-02	Bed allocation transaction	< 1 second
PERF-03	Discharge + waitlist check	< 1.5 seconds
PERF-04	Report generation from Views	< 3 seconds
PERF-05	Concurrent allocation safety	100% prevention of double-booking

8. Testing Strategy

8.1 Unit Testing (JUnit 5 + Mockito)

- Test DAO methods with in-memory H2 database.
- Test allocation logic with mocked `ResultSet`.
- Test priority score calculation.

8.2 Integration Testing

- Test complete admission → discharge → auto-allocation flow.
- Test concurrent allocation from two DAO instances (thread safety).
- Test trigger behavior: verify `bed_status_audit` records on admission/discharge.

8.3 Database Testing

- Verify all constraints (CHECK, FOREIGN KEY, UNIQUE).
- Verify views return correct aggregated data.
- Verify stored procedures handle edge cases (no available beds, invalid patient ID).

8.4 GUI Testing (Manual)

- Color coding renders correctly for all bed statuses.
- Form validation shows error dialogs for invalid input.
- Table sorting works on all columns.
- Background `SwingWorker` prevents UI freezing during DB calls.

9. Error Handling

Scenario	GUI Behavior	Database Behavior
DB connection lost	Show <code>JOptionPane.ERROR_MESSAGE</code> with retry option	Log error to console
Bed double-booking attempt	Show “Bed recently allocated to another patient”	Transaction rolls back
Invalid patient ID	Highlight field in red, show tooltip	Query returns empty, no action
No beds available	Show waitlist confirmation dialog	Patient added to <code>waitlist</code> table
Discharge of non-active admission	Show error dialog	Procedure returns error message

10. Deployment & Environment

10.1 Development Environment

- **OS:** Windows 11 / Ubuntu 22.04
- **IDE:** IntelliJ IDEA Community Edition
- **JDK:** OpenJDK 17
- **Database:** MySQL 8.0 Community Server
- **DB Client:** MySQL Workbench 8.0

10.2 Runtime Requirements

- Java Runtime Environment (JRE) 17+
- MySQL Server 8.0+ running locally or on LAN
- MySQL Connector/J JAR in classpath

10.3 Build Configuration (Maven `pom.xml` snippet)

```
<dependencies>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

10.4 Database Setup Script Execution Order

1. `schema.sql` — Create all tables with constraints
2. `indexes.sql` — Create performance indexes
3. `triggers.sql` — Create all triggers
4. `procedures.sql` — Create all stored procedures
5. `views.sql` — Create all views
6. `seed_data.sql` — Insert sample wards, beds, staff, patients

11. Version Control & Documentation

Document	Location	Format
Source Code	<code>/src</code>	Java files
SQL Scripts	<code>/sql</code>	<code>.sql</code> files

Table 6 – continued

Document	Location	Format
ER Diagram	/docs/er_diagram.png	PNG (from MySQL Workbench)
Normalization Doc	/docs/normalization.pdf	PDF
Project Report	/docs/report.pdf	PDF
README	/README.md	Markdown

12. Appendix: Seed Data Sample

```
-- Wards
INSERT INTO wards (ward_type, floor, capacity, daily_charge) VALUES
('General', 1, 10, 500.00),
('ICU', 2, 5, 2500.00),
('Emergency', 1, 8, 1000.00),
('Private', 3, 6, 1500.00);

-- Beds (20 total)
INSERT INTO beds (ward_id, bed_number, equipment_status, current_status) VALUES
(1, 'G-101', 'Standard', 'Available'),
(1, 'G-102', 'Standard', 'Available'),
(2, 'ICU-01', 'Ventilator, Monitor', 'Available'),
(2, 'ICU-02', 'Ventilator', 'Occupied'),
(3, 'E-101', 'Oxygen', 'Available'),
(4, 'P-301', 'Premium', 'Maintenance');

-- Staff
INSERT INTO staff (name, role, department, phone) VALUES
('Dr. Rajesh Sharma', 'Doctor', 'Cardiology', '9876543210'),
('Dr. Priya Patel', 'Doctor', 'General Medicine', '9876543211'),
('Nurse Anjali', 'Nurse', 'ICU', '9876543212');

-- Patients
INSERT INTO patients (name, age, blood_group, contact, address) VALUES
('Amit Kumar', 45, 'O+', '9123456789', 'Delhi'),
('Sunita Devi', 32, 'B+', '9123456790', 'Mumbai'),
('Rahul Singh', 67, 'A-', '9123456791', 'Bangalore');
```

End of TRD